

ACI · AIMLCZG557

NEAT, NAS & Neural Architecture Search

Advanced Computing for AI — Lesson 5

Exam: 28 Jun 2026 · Speciation + fitness sharing formula

How to use this companion. Blue = the idea. Amber = everyday analogy. Green = fully-solved example. Red = common mistake. Purple = one line to remember.

1 The Competing Conventions Problem

Standard GAs work with fixed-length chromosomes. For neural networks, the chromosome would be a list of weights. But here is the problem: two networks that compute the *same function* can have neurons numbered in a completely different order. Their weight vectors look totally different even though they are functionally identical. When you try to cross them over, you get a broken network — you are mixing weights from different connections.

The intuition

Imagine two teams playing the same tactical formation in football. Team A labels their midfielders 1,2,3. Team B labels theirs 3,1,2. They play identically, but if you create a “hybrid player” by averaging their jersey numbers, you get nonsense. NEAT’s solution is to label each player by when they *joined the club* (their innovation number), not by their position number.

Watch out

The competing conventions problem does NOT mean networks compute differently. Two networks can be functionally equivalent but have completely different weight-vector representations. This is why position-based crossover fails for neural networks.

Key takeaway

Standard GA crossover on neural networks is broken because two equivalent networks can look completely different in their weight vectors. NEAT solves this with innovation numbers.

2 NEAT: NeuroEvolution of Augmenting Topologies

NEAT (Stanley & Miikkulainen, 2002) solves the competing conventions problem with one idea: give every connection a **historical birth certificate** called an **innovation number**. Two genes with the same innovation number always represent the same structural connection, regardless of where they sit in the genome.

The intuition

Every time a new connection appears anywhere in the population, it gets a global serial number — its innovation number. When you align two genomes for crossover, you match genes by serial number. Same number = same connection. Different numbers = disjoint or excess genes.

2.1 NEAT Genome

A NEAT genome is a list of **connection genes**. Each gene has five fields:

Field	Meaning
In-Node	Source neuron ID
Out-Node	Destination neuron ID
Weight	Connection weight (real number)
Enabled	Is this connection currently active?
Innovation #	Global historical marker (assigned at birth)

2.2 NEAT Crossover

To cross two parents:

1. Align genes by innovation number.
2. **Matching genes** (same innovation #): inherit randomly from either parent.
3. **Disjoint genes** (in the range of the other parent but not matching): inherit from the more-fit parent.
4. **Excess genes** (beyond the range of the other parent): inherit from the more-fit parent.

2.3 NEAT Mutations

- **Weight mutation:** perturb a connection weight randomly.
- **Add connection:** pick two unconnected nodes; create a new connection with a new innovation number and a small random weight.
- **Add node:** pick an existing connection; split it by inserting a new node. The old connection is disabled; two new connections are added (each gets new innovation numbers). The incoming weight is set to 1.0, the outgoing weight to the old weight — this initialisation ensures the network behaves the same as before the mutation.

Where this shows up in ML. NEAT and its successors (HyperNEAT, DeepNEAT) are used to evolve neural architectures. Google's NASNet and Efficient-NAS used similar evolutionary search ideas, though typically with gradient-based training on a fixed topology.

3 Speciation & Fitness Sharing

NEAT faces another problem: a new structural mutation (like adding a node) initially makes things worse — it takes time to optimise. Without protection, the mutant is immediately outcompeted and eliminated. NEAT solves this with **speciation**: individuals are grouped into species based on structural similarity. They only compete within their own species.

The intuition

It is like an incubator for startups. A new company with a wild idea is protected from competition with established giants while it finds its feet. Once the new structure is optimised, it can compete globally.

Compatibility distance. The similarity between two genomes is measured by:

$$\delta = \frac{c_1 E + c_2 D}{N} + c_3 \cdot \bar{W}$$

where E = excess genes, D = disjoint genes, N = number of genes in larger genome (normalisation), $\bar{W}$ = average weight difference of matching genes. c_1, c_2, c_3 are tunable coefficients. Two individuals are in the same species if $\delta < \delta_t$ (the species threshold).

Fitness sharing. Within a species, each individual's fitness is divided by the number of individuals in that species:

$$f'_i = \frac{f_i}{|S_k|}$$

where $|S_k|$ is the size of individual i 's species. This prevents any one species from taking over the population.

Worked example: Fitness sharing calculation

Species S_1 has 4 individuals with raw fitness: 10, 8, 6, 4. Species S_2 has 2 individuals: 9, 7.

Step 1. Shared fitness in S_1 : $10/4 = 2.5$, $8/4 = 2.0$, $6/4 = 1.5$, $4/4 = 1.0$.

Step 2. Shared fitness in S_2 : $9/2 = 4.5$, $7/2 = 3.5$.

Step 3. Total shared fitness: S_1 contributes $2.5 + 2.0 + 1.5 + 1.0 = 7.0$; S_2 contributes $4.5 + 3.5 = 8.0$.

Step 4. S_2 will be allocated more offspring despite S_1 having the highest raw individual.

Insight: Fitness sharing favours species that are small but collectively strong over large species with mediocre average fitness.

Key takeaway

Speciation protects structural innovations. Fitness sharing ($f'_i = f_i/|S_k|$) prevents any species from monopolising the population.

4 DeepNEAT & CoDeepNEAT

Standard NEAT evolves individual connection weights in relatively small networks. **DeepNEAT** scales NEAT to deep networks by evolving *modules* rather than individual neurons. Each node in the NEAT graph represents an entire layer (e.g. a convolutional block).

CoDeepNEAT (co-evolution) evolves two separate populations simultaneously: a population of modules (individual layers) and a population of blueprints (how to assemble the modules). An assembled network = blueprint specifies which modules go where.

Where this shows up in ML. DeepNEAT and CoDeepNEAT were early milestones in Neural Architecture Search (NAS), showing that evolutionary methods could find competitive deep learning architectures without human design.

5 Neural Architecture Search (NAS)

NAS automates the design of neural network architectures. Instead of a human engineer choosing the number of layers, filter sizes, and connections, an algorithm searches over the space of possible architectures.

NAS Search Space. Defines what architectures are possible: layer types (conv, pooling, attention), connections (sequential, skip, branching), hyperparameters (filter size, stride).

NAS Search Strategy. How to navigate the search space:

- **Evolutionary** (NEAT-style): use a GA to evolve architecture genomes.

- **Reinforcement learning:** use a controller RNN to generate architecture descriptions; train with REINFORCE.
- **Gradient-based (DARTS):** make the architecture choice differentiable; use gradient descent on architecture parameters α .

DARTS (Differentiable Architecture Search). Instead of committing to one operation on each edge, DARTS mixes all operations with learned weights α :

$$\bar{o}^{(i,j)}(x) = \sum_{o \in \mathcal{O}} \frac{e^{\alpha_o^{(i,j)}}}{\sum_{o' \in \mathcal{O}} e^{\alpha_{o'}^{(i,j)}}} \cdot o(x)$$

The operation weights α are optimised by gradient descent alongside the network weights. At the end, discrete choices are made by taking $\arg \max_o \alpha_o$ for each edge.

✓ **Key takeaway**

NAS search strategies: evolutionary (GA), RL-controller, or gradient-based (DARTS). DARTS is the most efficient: it makes the architecture choice differentiable so both weights and architecture parameters are learned simultaneously.